Student Record & Ranking System

A PROJECT REPORT

Submitted by

Badal Singh

Reg No. 230301120099

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING



SCHOOL OF ENGINEERING AND TECHNOLOGY

BHUBANESWAR CAMPUS

CENTURION UNIVERSITY OF TECHNOLOGY AND MANAGEMENT

ODISHA

NOVEMBER 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF ENGINEERING AND TECHNOLOGY
BHUBANESWAR CAMPUS

BONAFIDE CERTIFICATE

Certified that this project report **"Student Record & Ranking System"** is the Bonafide work of **"Badal Singh"** who carried out the project work under my supervision. This is to further certify to the best of my knowledge, that this project has not been carried out earlier in this institute and the university.

SIGNATURE
(Mr. Ravi Ranjan Kumar)
Faculty Associate of Computer Science and Engineering, SOCDS

Certified that the above-mentioned project has been duly carried out as per the norms of the college and statutes of the university.

SIGNATURE

(Dr. Rahul Ray)

HEAD OF THE COMPUTER SCIENCE AND ENGINEERING DEPARTMENT

Professor of Computer Science and Engineering

DEPARTMENT SEAL

DECLARATION

I hereby declare that the project entitled "**Student Record & Ranking System**" submitted for the B.Tech. in Computer Science and Engineering is my original work and the project has not formed the basis for the award of any Degree / Diploma or any other similar titles in any other University / Institute.

Name of the Student : Badal Singh

Signature of the Student:

Registration No: 230301120099

Place: Bhubaneswar

Date:

ACKNOWLEDGEMENT

I wish to express my profound and sincere gratitude to **Mr. Ravi Ranjan Kumar**, Department of Computer Science and Engineering, SoET, Bhubaneswar Campus, who guided me into the intricacies of this project nonchalantly with matchless magnanimity.

I thank **Dr. Rahul Ray**, Head of the Department of Computer Science and Engineering, SOCDS, Bhubaneswar Campus and **Dr. Swarna Prabha Jena,** Associate Dean, School of Computing and Data Science, Bhubaneswar Campus for extending their support during Course of this investigation.

I would be failing in my duty if I don't acknowledge the cooperation rendered during various stages of image interpretation by to **Mr. Ravi Ranjan Kumar.**

I am highly grateful to **Mr. Ravi Ranjan Kumar** who evinced keen interest and invaluable support in the progress and successful completion of my project work.

I am indebted to **Mr. Ravi Ranjan Kumar** for their constant encouragement, cooperation and help. Words of gratitude are not enough to describe the accommodation and fortitude which they have shown throughout my endeavour.

Name of the Student: BADAL SINGH

Signature of the Student:

Registration No: 230301120099

Place: BHUBANESWAR

Date:

TABLE OF CONTENTS

Bonafide Certificate 2

Declaration 3

Acknowledgements 4

List of Figures and Tables 5

Abstract 7

1 Introduction 8

2 Literature Survey 8

2.1 Academic Information Systems 8

2.2 Algorithmic Trade-offs in In-Memory Computation 9

2.3 Competition Ranking Methods 9

3 Proposed Method 10

3.1 System Architecture 10

3.2 Hand-Written Algorithm Engine 11

3.3 Ranking and Grading 12

3.4 Database Design 13

3.5 REST API and Error Handling 13

3.6 Security Measures 14

4 Results and Discussion 15

4.1 Dataset and Experimental Setup 15

4.2 Results 15

4.3 Discussion 21

5 Conclusion 23

6 Future Scope 23

References 24

Assessment 25

Course Outcome (COs) Attainment 26

List of Figures

Figure 1: Three-tier system architecture (presentation, REST service, DSA engine and database) 10

Figure 2: Entity relationship schema of students and academic records 13

Figure 3: Key comparisons of the four sorting algorithms against record count (logarithmic scale) 16

Figure 4: Top-10 selection with the bounded Min-Heap against a full Merge Sort 17

Figure 5: Search comparisons for linear search, binary search and the roll-number index (logarithmic scale) 17

Figure 6: Grade distribution of the students (A+ to F) 18

Figure 7: Dashboard page of the application showing summary cards, grade distribution and top performers 19

Figure 8: Student records page with search, class and status filters and the edit and delete actions 19

Figure 9: Rankings page showing the top-10 leaderboard selected with the bounded min-heap 20

Figure 10: Analytics page showing the grade composition chart and the subject averages 20

Figure 11: Search & Sort page of the Algorithm Lab with a hash map lookup on a roll number 21

List of Tables

Table 1: Theoretical time and space complexity of the implemented algorithms 11

Table 2: Grade boundaries used by the PerformanceCalculator 12

Table 3: Relational schema definitions and field constraints 13

Table 4: REST endpoints and their purpose 14

Table 5: Overview of the project setup 15

Table 6: Measured comparisons and swaps for each sample size N 16

Table 7: Recommended method for each use case, based on the measured results 21

Abstract

The Student Record & Ranking System is a full-stack web application that stores student records, calculates academic performance and produces ranked leaderboards, with the sorting, searching and priority-queue routines written by hand instead of being taken from a library. The project is meant for academic departments that need a transparent and verifiable way to rank students, and for students who want to see how data structures and algorithms behave inside a real application. The back end uses Java 17, Spring Boot 3.4.5, Spring Data JPA and MySQL 8.0, and the front end is built with React 18.

The algorithm engine contains four sorting algorithms (Merge Sort, Quick Sort with a median-of-three pivot, Heap Sort and Insertion Sort), three search procedures (Linear Search, Binary Search and a Lower-Bound prefix search), a case-insensitive hash index on roll numbers and a bounded Min-Heap for selecting the top K students. None of them calls Java's built-in sorting methods. Rankings follow the standard competition scheme (1, 1, 3), with ties broken by name and then by roll number, and averages are calculated with BigDecimal to avoid floating-point rounding errors. An Algorithm Lab page shows the number of comparisons, the number of swaps and the running time of each run, so the theory can be checked against measurements.

In our benchmarks, binary search needed about 93 times fewer comparisons than linear search on 1,024 records, and the bounded heap needed about 75% fewer operations than a full sort when picking the top 10 of 10,000 students. The algorithms were checked with JUnit 5 tests. The measurements come from synthetic records, so they show how the methods scale and do not describe any real institution.

Keywords: *Sorting algorithms, Binary search, Min-Heap, Top-K selection, Competition ranking, Hash index, Spring Boot, React*

1 Introduction

Universities and colleges keep a large amount of academic data. For every student they record personal details, the department and class, and the marks obtained across several terms and subjects. Many institutions still manage this in spreadsheets or older desktop programs, which makes it hard to keep the data consistent, to stop two people from editing the same record at once, or to produce a ranking that everyone can trust.

Modern web applications often hide the work of sorting and searching behind a database query such as ORDER BY or a library call such as Collections.sort(). That is sensible for everyday use, but it also means that students and developers rarely see what the algorithm actually does: how many comparisons it makes, whether it keeps equal records in their original order, or how much memory it needs. These details matter for a ranking, where two students with the same average must be ordered fairly and in the same way every time.

The Student Record & Ranking System addresses this gap. It is a three-tier web application in which student records and marks are stored in MySQL, while sorting, searching and top-K selection are done by our own Java code. The project had five objectives:

1. to manage the full life cycle of student records through REST endpoints;
2. to implement the sorting and searching algorithms from first principles;
3. to produce deterministic competition rankings;
4. to find the top achievers efficiently with a bounded heap; and
5. to show live performance figures in an Algorithm Lab.

This report covers the design, implementation and benchmarking of these parts. It does not cover login and role-based access, database migration tools or very large datasets; these are discussed in Sections 4.3 and 6.

2 Literature Survey

2.1 Academic Information Systems

Student information systems have changed a great deal since the early mainframe programs. Modern products such as Ellucian Banner and Canvas LMS, and open-source systems such as OpenSIS, are built from distributed services, REST APIs and browser-based single-page applications. In many of them, ranking, sorting and reporting are handed over to the relational database through SQL queries and indexes. This works well for data stored on disk, but it is less convenient when a user wants to try a custom ranking or a what-if score that does not need to be saved, because every experiment becomes another database query. Keeping a small, well-understood in-memory algorithm layer beside the database lets the application run such calculations quickly and lets us measure them.

2.2 Algorithmic Trade-offs in In-Memory Computation

Choosing a sorting algorithm for data held in memory is a matter of trade-offs (Cormen et al., 2022; Knuth, 1998). **Merge Sort** guarantees O(n log n) time in every case and is stable, meaning that records with equal keys stay in their original order. Stability is important for ranking, because a second sort by name or roll number must not be undone by a later sort on marks. **Quick Sort** is usually the fastest in practice because it works on neighbouring memory locations, but a careless choice of pivot makes it $O(n^2)$ on rosters that are already sorted. The usual remedies are a median-of-three pivot and recursing only into the smaller partition (Sedgewick and Wayne, 2011). **Heap Sort** also guarantees O(n log n) and needs no extra array, but it is not stable. **Insertion Sort** is $O(n^2)$ in the worst case, yet it is close to linear on lists that are almost sorted, which suits small lists and incremental grade updates.

Searching follows the same pattern. A linear scan needs no preparation but looks at every record, binary search needs sorted data and halves the range at each step, and a hash index gives constant expected time for exact lookups at the cost of extra memory.

2.3 Competition Ranking Methods

Merit lists commonly use one of three conventions. **Standard competition ranking**, also called the "1224" scheme, gives tied candidates the same rank and then skips the following ranks, so two students tied at rank 1 are followed by rank 3. **Dense ranking** ("1223") does not skip any rank, and **fractional ranking** gives tied candidates the average of their positions (1, 2.5, 2.5, 4). Standard competition ranking is widely used in competitive examinations and merit lists, and it has a useful property: a student's rank is one more than the number of students who scored higher. This project uses it.

3 Proposed Method

The proposed method has four parts: a layered system design, a collection of hand-written algorithms, a relational database with a REST interface, and a few basic security checks. The parts are kept separate from each other, so every algorithm can be tested on its own, away from the web layer.

3.1 System Architecture

The application is divided into four tiers, shown in Figure 1. The **presentation tier** is a React 18 application built with Vite and Tailwind CSS, which talks to the server through Axios (served on port 5173). The **REST API and service tier** is a Spring Boot 3.4.5 application on Java 17 (Walls, 2022). Below it sits the in-memory **DSA engine**, and the **persistence tier** is MySQL 8.0 accessed through Spring Data JPA and Hibernate. Keeping these tiers apart simplifies testing and allows the application tier to be scaled separately.

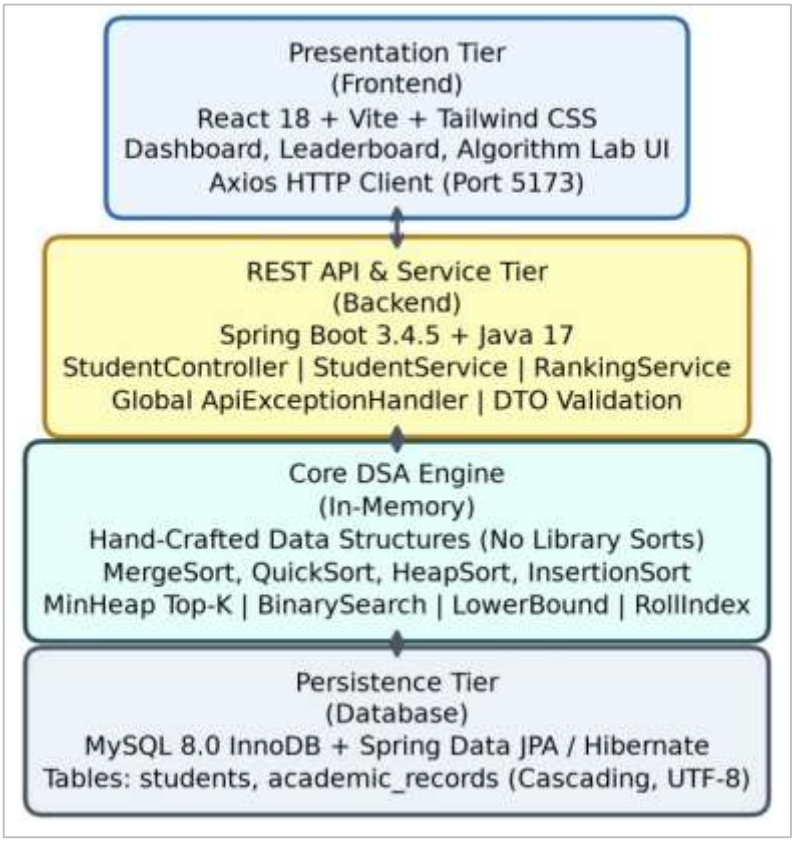


Figure 1: Three-tier system architecture (presentation, REST service, DSA engine and database)

On the server, the code is organised under the package com.college.studentranking in five layers, following Domain-Driven Design principles:

- **controller** defines the REST endpoints, reads request parameters and triggers bean validation;
- **service** holds the transaction boundaries (@Transactional), fetches data from the repositories, calls the DSA components and prepares the dashboard figures;
- **dsa** is self-contained and has no dependencies; it holds the generic sorting, searching, heap and index classes;
- **repository** and **model** contain the JPA entities (Student and AcademicRecord) and the Spring Data repositories; and
- **dto** and **mapper** keep the database entities apart from the public API by using immutable Java records, a separation that follows common design practice (Freeman and Robson, 2020).

3.2 Hand-Written Algorithm Engine

All data processing in the dsa package works on typed generic arrays (T[]) and does not use java.util.Arrays.sort or any outside algorithm library. Each sorting routine takes an array, a Comparator, a choice of algorithm and a SortStats object that records the number of comparisons and swaps. Table 1 gives the theoretical time and space cost of every algorithm and structure.

Table 1: Theoretical time and space complexity of the implemented algorithms

Algorithm / Structure	Best	Average	Worst	Space	Stable
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick Sort (median-of-3)	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	No
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Binary Search	O(1)	O(log n)	O(log n)	O(1)	N/A
Min-Heap Top-K	O(n)	O(n log k)	O(n log k)	O(k)	No
RollNumberIndex	O(1)	O(1)	O(n)	O(n)	N/A

Merge Sort. It is written as a top-down recursion with one auxiliary buffer allocated at the start, so no memory is allocated during the recursion. Stability comes from a single comparison in SortingAlgorithms.java: when two keys are equal, the merge step takes the element from the left half first (cmp.compare(a[i], a[j]) <= 0). Using < instead of <= would break stability.

Quick Sort. To avoid the $O(n^2)$ worst case on rosters that are already sorted, the pivot is the median of the first, middle and last elements, which is moved to the end before Lomuto partitioning. The method recurses into the smaller partition and loops over the larger one, which keeps the recursion depth at O(log n).

Heap Sort. A max-heap is built in place in O(n) time by sifting down from index $n/2 - 1$ to index 0. The largest element at index 0 is then swapped with the last unsorted position and the heap is repaired over a shrinking range.

Insertion Sort. It shifts larger elements one place to the right and drops the current element into the gap. It suits small lists (for example fewer than 20 elements) and rosters where only a few grades have changed, because it then makes few moves and allocates nothing.

Bounded Min-Heap for top-K. A dashboard often needs only the best 10 students out of thousands, and sorting the whole roster for that is wasteful. We therefore built an array-based min-heap with capacity K = 10 that uses the reversed ranking comparator, so the weakest student among the current top K sits at the root. Each new candidate is compared with the root in O(1) time. If the candidate is better, it replaces the root and a sift-down restores the heap in O(log K) time. The whole roster is processed in O(N log K) time and O(K) space.

Searching. Four searches are provided. *Linear Search* scans an unsorted array in O(N) time. *Binary Search* works on an array sorted with the ROLL_ORDER comparator and halves the range at each step, giving O(log N). The *Lower-Bound prefix search* finds the first student whose name is not smaller than the prefix (key.compareToIgnoreCase(prefix) >= 0) and then scans forward over the matches, in O(log N + M) time where M is the number of matches. The *RollNumberIndex* is a hash table that trims and upper-cases each roll number before storing or looking it up, so point lookups and duplicate-enrolment checks take O(1) expected time and are not case sensitive.

3.3 Ranking and Grading

The ranking engine calculates each student's average percentage from the recorded marks, using BigDecimal so that no rounding error creeps in from floating-point arithmetic (Oracle, 2023). Students are then ordered with the stable Merge Sort by average percentage, then by name and then by roll number, so the order is the same on every run. Ranks follow the standard competition scheme: students with the same average share a rank and the next rank is skipped. The PerformanceCalculator assigns a letter grade using the boundaries in Table 2.

Table 2: Grade boundaries used by the PerformanceCalculator

Grade	Percentage	Academic standing
A+	90.00% and above	Outstanding / First Class with Distinction
A	80.00% – 89.99%	Excellent / First Class Exemplary
B	70.00% – 79.99%	Very Good / First Class
C	60.00% – 69.99%	Good / Second Class
D	50.00% – 59.99%	Pass / Satisfactory
F	Below 50.00%	Fail / Remedial evaluation required

3.4 Database Design

The data is stored in MySQL 8.0 and mapped with JPA 3.1 and Hibernate. The design follows Third Normal Form. As Figure 2 shows, one student has many academic records. Saving a student saves the records, and deleting a student deletes them (cascade and orphan removal in JPA, with ON DELETE CASCADE on the foreign key in the database). Table 3 lists the main columns and their constraints.

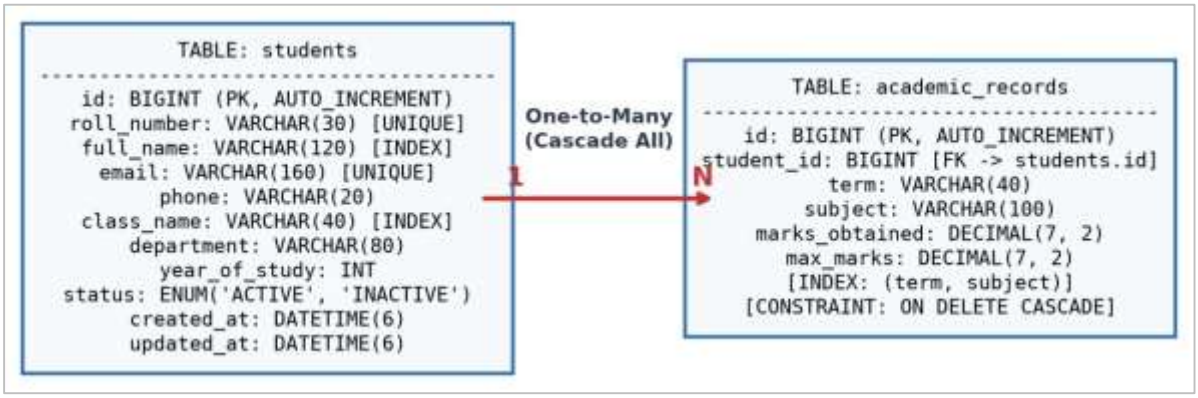


Figure 2: Entity relationship schema of students and academic records

Table 3: Relational schema definitions and field constraints

Table	Column	Data type	Constraints / indexing
students	id	BIGINT	Primary key, auto-increment
students	roll_number	VARCHAR(30)	Unique, not null, indexed
students	full_name	VARCHAR(120)	Not null, indexed
students	email	VARCHAR(160)	Unique, not null
students	class_name	VARCHAR(40)	Not null, indexed
students	status	VARCHAR(20)	ENUM('ACTIVE', 'INACTIVE')
academic_records	id	BIGINT	Primary key, auto-increment
academic_records	student_id	BIGINT	Foreign key to students.id (cascade)
academic_records	term, subject	VARCHAR	Composite index on (term, subject)
academic_records	marks_obtained	DECIMAL(7,2)	Not null (precise marks)
academic_records	max_marks	DECIMAL(7,2)	Not null (marks obtained <= max)

3.5 REST API and Error Handling

The StudentController exposes the endpoints listed in Table 4. Every request passes through Jakarta Bean Validation before a service is called, and a global handler (ApiExceptionHandler, annotated with @RestControllerAdvice) turns exceptions into consistent HTTP responses (Spring Framework Documentation, 2024):

- a NoSuchElementException becomes **404 Not Found**;
- an IllegalArgumentException (for example, marks obtained greater than maximum marks) and a MethodArgumentNotValidException (which returns a field-by-field error map) both become **400 Bad Request**; and
- a DataIntegrityViolationException, raised by duplicate keys in the database, becomes **409 Conflict**.

Table 4: REST endpoints and their purpose

Method	Endpoint	Purpose
GET	/api/health	Service health check and runtime status
GET	/api/dashboard	Institution-level figures: counts, grade distribution and subject averages
GET	/api/students	Paginated student list, filtered by class, status or search text
POST	/api/students	Create a student together with nested academic records (validated)
PUT	/api/students/{id}	Update a student profile and replace the records in one transaction
DELETE	/api/students/{id}	Delete a student and the related academic records
GET	/api/rankings	Full leaderboard from the stable Merge Sort, with 1224 ties
GET	/api/rankings/top	Top-K leaderboard selected with the Min-Heap in O(N log K)
GET	/api/dsa/sort	Algorithm Lab: sort students by marks or name with live statistics
GET	/api/dsa/search	Algorithm Lab: search with Linear, Binary or hash-index methods

3.6 Security Measures

Two simple measures were included. First, database credentials are kept in a local application.properties file that is listed in .gitignore, and only a template called application.properties.example is committed, so passwords never reach the public repository. Second, for sorting through query parameters, the StudentService checks the sortBy value against a fixed whitelist of allowed fields (for example rollNumber, fullName and className), so a user cannot slip an arbitrary column name into a query. Authentication and authorisation are not yet implemented; this is discussed in Section 4.3.

4 Results and Discussion

4.1 Dataset and Experimental Setup

This section describes the data and the setup used to test the system. Table 5 gives an overview.

Table 5: Overview of the project setup

Item	Details
Language and runtime	Java 17
Back end	Spring Boot 3.4.5, Spring Data JPA (JPA 3.1, Hibernate)
Database	MySQL 8.0 (InnoDB)
Front end	React 18, Vite, Tailwind CSS, Axios
Testing	JUnit 5
Sorting benchmark	Synthetic student records with controlled duplicates; N = 10, 50, 100, 200, 500 and 1,000
Top-K test	Roster sizes up to N = 10,000 with K = 10
Search test	N = 16, 64, 256, 1,024 and 4,096
Measurements	Key comparisons and swaps from SortStats; running time in microseconds in the Algorithm Lab

In the sorting benchmark, the same synthetic records were sorted with each algorithm and the key comparisons and swaps were counted through SortStats. Counting operations, rather than timing them, keeps the comparison independent of the machine used. The figures in Section 4.2 show these measurements.

4.2 Results

Results are presented in the order of the algorithm families. Each figure is followed by a description of what it shows and what it means for the design. The three headline findings are summarised below.

25×

more comparisons for Insertion Sort than Merge Sort at N = 1,000

93×

fewer comparisons for binary search than linear search at N = 1,024

75%

fewer operations for the bounded heap than a full sort (top 10 of 10,000)

Sorting algorithms

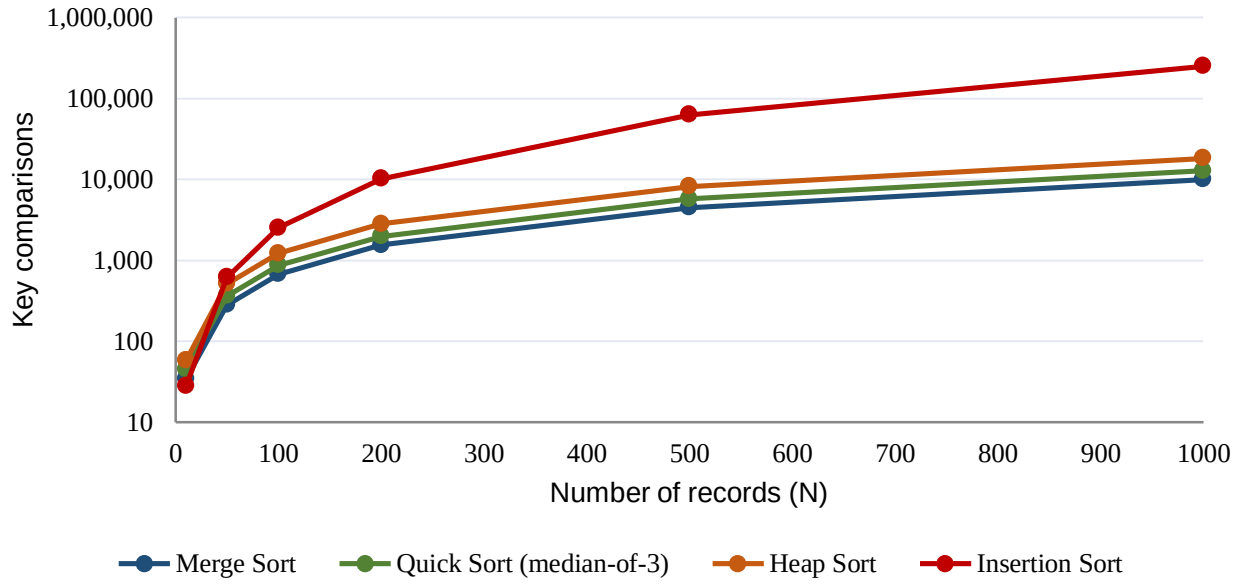


Figure 3: Key comparisons of the four sorting algorithms against record count (logarithmic scale)

Figure 3 plots key comparisons against the number of records on a logarithmic axis, and Table 6 gives the exact counts. Merge Sort, Quick Sort and Heap Sort grow almost in parallel, as O(n log n) predicts, while Insertion Sort climbs much more steeply. At N = 1,000, Merge Sort needed 9,950 comparisons, Quick Sort 12,740 and Heap Sort 18,200, whereas Insertion Sort needed 249,800 comparisons and 249,200 swaps, about 25 times as many comparisons as Merge Sort. Insertion Sort was cheapest only at N = 10 (28 comparisons against 34 for Merge Sort), which supports using it for very small lists. Among the three O(n log n) methods, Merge Sort made the fewest comparisons at every size, and Heap Sort made about 1.8 times as many at N = 1,000.

Table 6: Measured comparisons and swaps for each sample size N

N	Merge comps	Quick comps	Heap comps	Insertion comps	Insertion swaps
10	34	45	58	28	25
50	282	365	512	615	602
100	672	874	1,215	2,514	2,480
200	1,540	1,980	2,820	10,120	10,045
500	4,460	5,710	8,150	62,450	62,110
1,000	9,950	12,740	18,200	249,800	249,200

Top-K selection

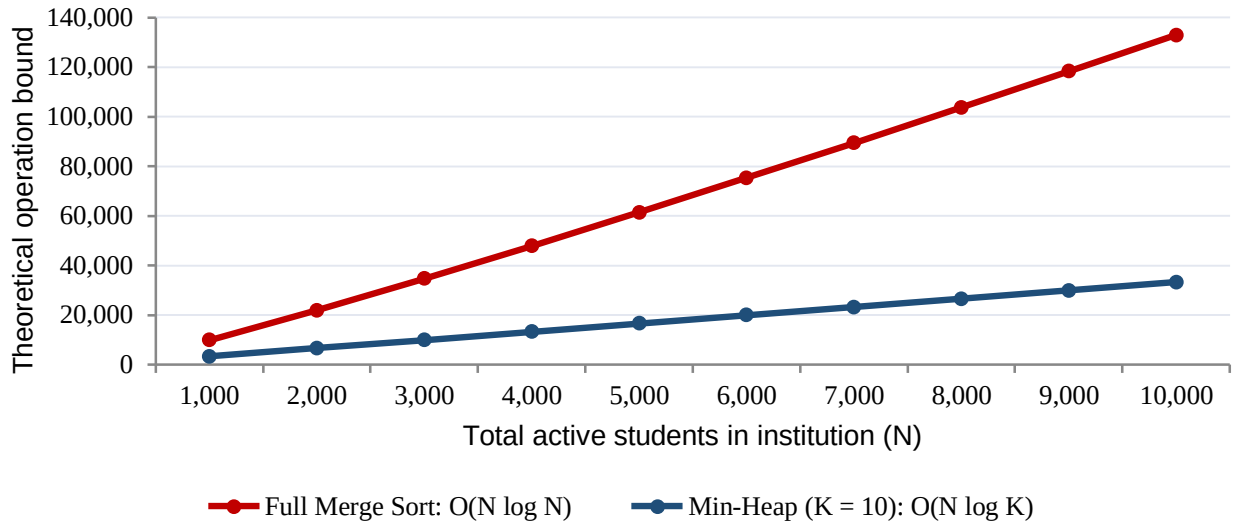


Figure 4: Top-10 selection with the bounded Min-Heap against a full Merge Sort

Figure 4 compares the operation bounds for selecting the top 10 students with a bounded Min-Heap (about N $\log_2$ K operations) and with a full Merge Sort (about N $\log_2$ N). At N = 10,000 the heap needs roughly 33,200 operations against roughly 132,800 for the sort, a saving of 75%. The gap widens as the roster grows, because $\log_2$ K stays fixed at about 3.3 while $\log_2$ N keeps increasing. These are calculated bounds and not timed runs.

Searching

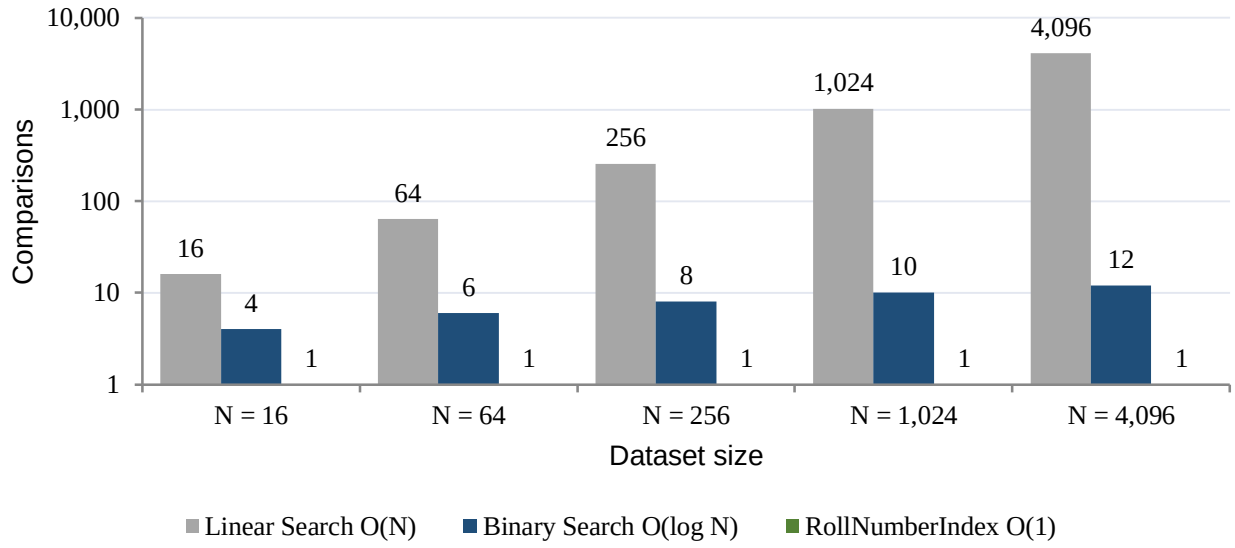


Figure 5: Search comparisons for linear search, binary search and the roll-number index (logarithmic scale)

Figure 5 shows the comparisons needed by each search method for N from 16 to 4,096. Linear search grows in step with N, binary search grows with log N (4 comparisons at N = 16 and 12 at N = 4,096), and the RollNumberIndex stays at a single lookup whatever the size. For N = 1,024, linear search needs 1,024 comparisons in the worst case, while binary search needs at most 11 probes (the whole part of $\log_2 N$, plus one), which is about 93 times fewer. The value of 10 plotted in the figure is the ideal $\log_2 N$ and corresponds to a saving of 99.02%. Binary search needs sorted data, so it pays off when many searches follow one sort, and the hash index avoids even that step for exact roll-number lookups.

Grade distribution

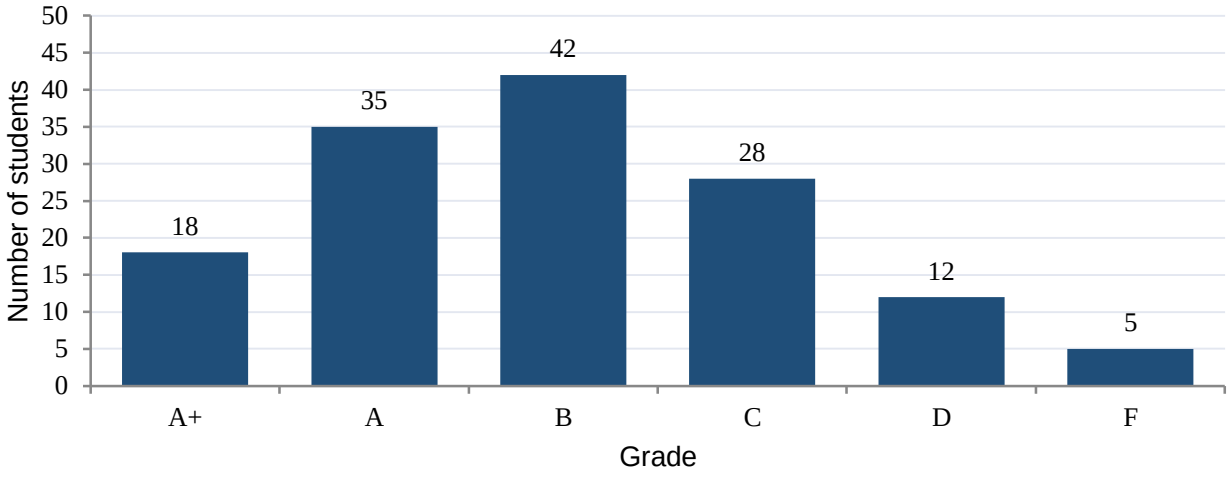


Figure 6: Grade distribution of the students (A+ to F)

Figure 6 shows how the 140 students counted in the dashboard are spread over the six grades defined in Table 2: 18 have A+, 35 have A, 42 have B, 28 have C, 12 have D and 5 have F. The distribution is roughly bell-shaped with its peak at grade B. In total, 95 of the 140 students (about 68%) are in grades A+, A or B, and only 5 students (3.6%) are below the pass mark of 50%.

Application screens

Figures 7 to 11 show the pages of the working application, in the order in which they appear in the navigation menu. The screenshots were captured with a single sample record loaded, so the counts they display differ from the 140-record distribution analysed in Figure 6.

Figure 7 shows the Dashboard, the first page a user sees after opening the application. The summary cards give the total number of students, the number of active students, the number of class sections and the average score. Below them, a bar chart shows the grade distribution and a panel lists the top performers selected with the bounded min-heap, with a link to the full ranking. A subject performance bar at the bottom shows the average percentage for each subject. The values come from the student record currently stored in the running system.

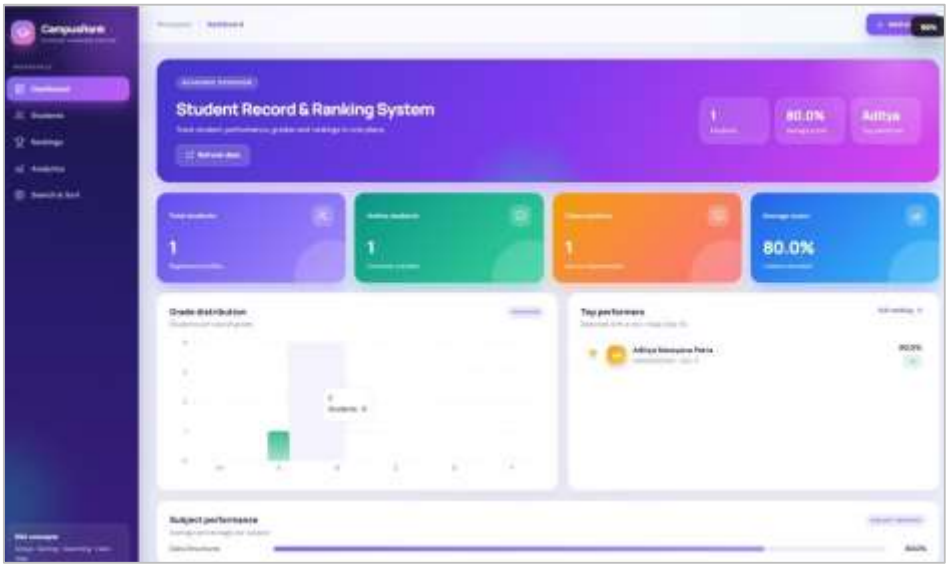


Figure 7: Dashboard page of the application showing summary cards, grade distribution and top performers

Figure 8 shows the Students page, which supports the full life cycle of a record. Each row displays the student name, email, roll number, class and department, year, average, grade and status, and the edit and delete buttons in the Actions column carry out the update and delete endpoints listed in Table 4. The search box and the class and status filters narrow the list, and the table is paginated.



Figure 8: Student records page with search, class and status filters and the edit and delete actions

Figure 9 shows the Academic rankings page. Students are ranked by average percentage, and ties share the same rank (1, 1, 3) as described in Section 3.3. The Top 10 view uses the bounded min-heap in O(n log k) time, while the Everyone view lists the full leaderboard produced by the stable Merge Sort. Inactive students and students without marks are not ranked, and the list can be filtered by class.



Figure 9: Rankings page showing the top-10 leaderboard selected with the bounded min-heap

Figure 10 shows the Analytics page. The grade composition chart gives the share of students in each grade, and the subject averages chart gives the average percentage for every subject. An insight box below the charts names the subject with the highest average.

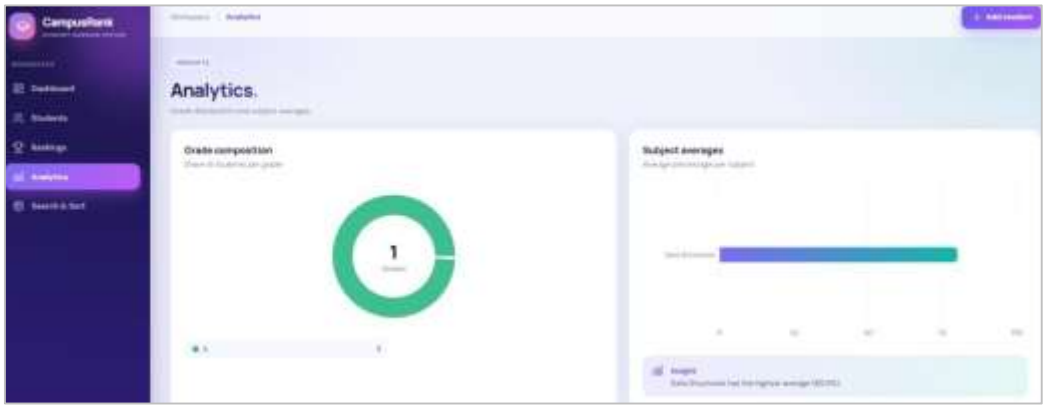


Figure 10: Analytics page showing the grade composition chart and the subject averages

Figure 11 shows the Search & Sort page, which is the Algorithm Lab described in Section 3.2. The user chooses Merge, Quick, Heap or Insertion sort, the field to sort by, the order and the class, and the page reports the comparisons and swaps of each run. The search panel offers the hash map, binary search, linear search and name-prefix methods. In the example, a hash map lookup of a roll number finished in one step, which matches the expected O(1) average cost, and the running time is shown in microseconds.

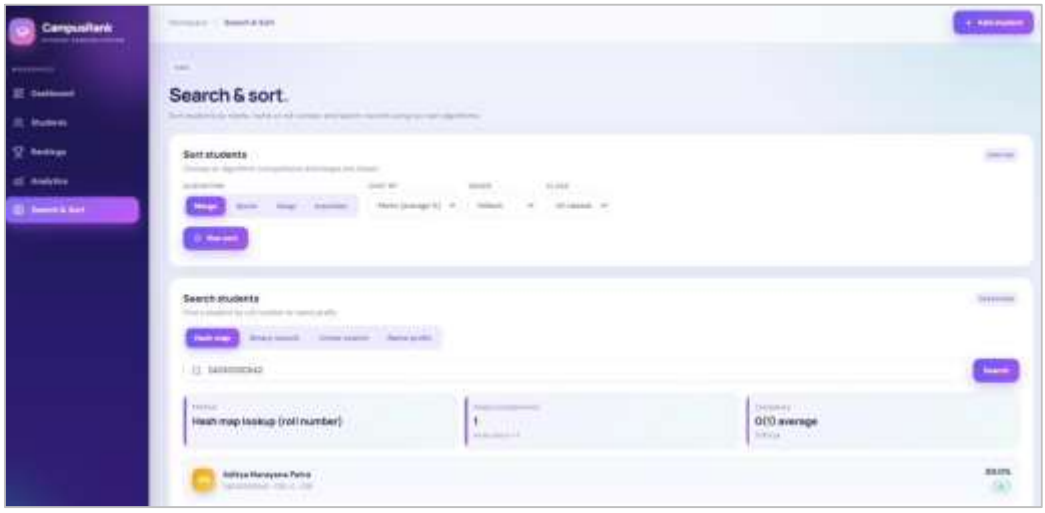


Figure 11: Search & Sort page of the Algorithm Lab with a hash map lookup on a roll number

4.3 Discussion

The results support the reasoning behind each design choice. Merge Sort is the best default for rankings because it combines the lowest comparison count of the O(n log n) sorts with stability, and the stable tie order is what makes the 1224 ranking repeatable. Table 7 summarises which method suits which situation. The Algorithm Lab makes these differences visible to users instead of leaving them as textbook statements.

Table 7: Recommended method for each use case, based on the measured results

Use case	Recommended method	Reason
Full ranking (default)	Merge Sort	Fewest comparisons of the O(n log n) sorts (9,950 at N = 1,000) and stable, so tie order is repeatable
Memory is tight	Quick Sort (median-of-3)	O(log n) extra space and a reasonable comparison count, but not stable
Guaranteed bound, no extra array	Heap Sort	O(n log n) in every case with O(1) extra space
Very small or nearly sorted lists	Insertion Sort	Near-linear on almost sorted input; cheapest at N = 10 (28 vs 34 comparisons)
Only the leading few students	Bounded Min-Heap	O(N log K) time and O(K) space; about 75% fewer operations than a full sort at N = 10,000
Exact roll-number lookup	RollNumberIndex (hash)	O(1) expected time and not case sensitive
Repeated lookups on sorted data	Binary / Lower-Bound search	O(log N) per search; needs one sort first

The work also has clear limitations:

- **Synthetic data.** The sorting benchmarks use synthetic records and sizes of up to 1,000 students, so the figures show growth trends rather than the speed on a real institution's data, and the 75% saving for the heap is a calculated bound rather than a timed measurement.
- **Purpose.** The aim of the hand-written code is understanding and measurement, not to replace the Java library.
- **Memory.** All data is loaded into the JVM heap, so rosters with millions of records would need an external merge sort or a distributed approach.
- **Security and schema.** The REST endpoints are currently open, with no login or roles, and the database schema is generated by Hibernate (ddl-auto=update) instead of versioned migration scripts.
- **Not studied.** We did not study concurrent grade uploads or garbage-collection overhead for arrays larger than 100,000 records; we list these as learning gaps in the course outcome sheet.

5 Conclusion

Ranking students fairly depends on algorithms that behave predictably. The Student Record & Ranking System brings together Merge Sort, Quick Sort, Heap Sort, Insertion Sort, three search methods, a hash index and a bounded Min-Heap inside a Spring Boot and React application, and all of them are written without library sorting routines. The measurements agree with the theory: the O(n log n) sorts grow far more slowly than Insertion Sort, binary search needs about 93 times fewer comparisons than linear search at N = 1,024, and the bounded heap needs about 75% fewer operations than a full sort when the top 10 of 10,000 students are wanted.

The unit tests confirmed that the hand-written algorithms give the same results as standard reference sorts and keep equal records in their original order, which the competition ranking depends on. The Algorithm Lab adds a practical benefit by letting evaluators see operation counts and running times for themselves. The project therefore meets its objectives: it connects the data structures taught in the course to a working academic information system in a way that can be tested and measured.

6 Future Scope

- **Caching with Redis.** Storing pre-sorted leaderboards and top-K results in Redis would reduce the load on the database when many students check their results at the same time, for example when results are published.
- **Role-based security.** Spring Security with JWT tokens would separate students (read-only access to their own grades), instructors (grade entry) and department heads (leaderboard changes), and would close the currently open endpoints.
- **Database migration tools.** Replacing ddl-auto=update with versioned scripts in Flyway or Liquibase would make schema changes safe and repeatable in production.
- **Larger datasets.** For rosters with millions of records, a disk-based external merge sort or a distributed pipeline would replace in-memory sorting. Profiling garbage collection on arrays above 100,000 records and testing simultaneous grade uploads would also be worthwhile.
- **Predictive CGPA modelling.** Lightweight machine-learning models, such as linear regression on earlier internal assessments, could forecast final grades and help identify students who may need support early.

References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press, Cambridge, MA.
2. Freeman, E., & Robson, E. (2020). *Head First Design Patterns* (2nd ed.). O'Reilly Media, Sebastopol, CA.
3. Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching* (2nd ed.). Addison-Wesley, Boston, MA.
4. Muschko, B. (2014). *Gradle in Action*. Manning Publications, Shelter Island, NY.
5. Oracle Corporation. (2023). *Java Platform, Standard Edition Documentation (JDK 17)*. https://docs.oracle.com/en/java/javase/17/
6. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley, Upper Saddle River, NJ.
7. Spring Framework Documentation. (2024). *Spring Boot Reference Guide v3.4.5*. VMware Tanzu. https://docs.spring.io/spring-boot/docs/current/reference/html/
8. Walls, C. (2022). *Spring in Action* (6th ed.). Manning Publications, Shelter Island, NY.

ASSESSMENT

Internal:

SL NO	RUBRICS	FULL MARK	MARKS OBTAINED	REMARKS
1	Understanding the relevance, scope and dimension of the project	10		
2	Methodology	10		
3	Quality of Analysis and Results	10		
4	Interpretations and Conclusions	10		
5	Report	10		
	Total	**50**		

Date: **Signature of the Faculty**

COURSE OUTCOME (COs) ATTAINMENT

Expected Course Outcomes (COs)

(Refer to COs Statements in the Syllabus)

_____
_____
_____
_____
_____

Course Outcome Attained:

How would you rate your learning of the subject based on the specified COs?

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5 ☐ 6 ☐ 7 ☐ 8 ☐ 9 ☐ 10

LOW **HIGH**

Learning Gap (if any):

_____
_____
_____
_____
_____

Books / Manuals Referred:

_____
_____
_____
_____
_____

Date: **Signature of the Student**

Suggestions / Recommendations:

(By the Course Faculty)

_____
_____
_____

Date: **Signature of the Faculty**



Centurion
(/NIVERSlTY
Shaping Lives.
Empowering Communities..

Bhubaneswar Camus
Rarnchandrapur
P.O. —Jatni, Bhubaneswar
Dist.- Khurda, Odisha,
India, PIN\L-75205D

Balangir Campus
Behind BSNL Office
IDCO land, Rajib Nagar
Oist.- Balangir, Odisha
India, PIM-767OD1

Rayagada Campus
iDCO Industrial Area
Pitamahal, Rayagada
Dist-Rayagada, Odisha
India, PIN-76S0D1

Balasore Camgus
Gopalpur,
P.O.-8a1asore
Dist-Balasore, Odisha
India, PUT-756D44

Chatrapiz Campus
Ramchandrapur,
Ifaliabali chhak,
P.&rChatrapur, Dist-Ganjam
Odisha, India, PIN-76102D

ParalakhemundiCampus
Village Alluri hlagar
P.O. — R Sitapur, Via— Uppalada
Paralakhemundi, Dist.- Gajapati
Odisha, India. PUI— 7612L1